

Dual-sparse weight-stationary SNN array: full report and handoff

Written for the next Claude (and for the author). Simple words on purpose. Everything here was built and run in the folder with this file. Files are listed at the end.

0. The short version

- The design is **Mentha + weight-stationary + 8 timesteps in parallel + one-cycle IF**.
- I built a **cycle-stepped simulator** of it (`snn_sim.py`). It moves every number through the 4x4 array one cycle at a time.
- Its output spikes are **bit-exact equal** to a plain dense calculation, on the 64x16 * 16x128 example, on random layers, and on a 3-layer chain. So running a network on it gives the same answers a dense/LoAS-style calculation gives.
- For whole networks I use a **fast model** (random sampled slabs, same rules). It matches the stepped simulator within 2.5% on average (worst 8.5%).
- **UPDATE (latest, read section 15 first):** the author then said both weights and activations are 90%+ sparse. With that, Ours is **1.4x to 3.6x faster at batch 1 and 4.2x to 5.0x higher throughput at batch 16**, and the gain grows with sparsity. The bullet below used the table's combined-sparsity numbers and is kept for history.
- **Result against LoAS** (using the layers and sparsity numbers in `cycle_compare.py`): roughly a **tie** on AlexNet and ResNet18, and **slower** on VGG16 (2.2x to 2.4x at batch 1). At batch 16 the throughput is within about -3% to +18% of LoAS. This is NOT a clear win. Section 9 says why and what could change it.
- **Throughput is not 1/latency.** LoAS stays flat when the batch grows. Ours climbs a lot. Section 8 shows the derivation.

1. Names (final)

C = A x B

Name	What it is	Shape	Where it lives
A	weights, int8	M x K	stationary, inside the PEs
B	input activations, one 8-bit mask per element (bit t = spike at timestep t)	K x N	streams through the array
C	output, 8 spike bits per element (the next layer's B). Before IF it is a psum element: 8 timesteps x 16-bit sums = 128 bits in the array, 8 x 24 = 192 bits in the partial buffer	M x N	

The example in the chat, "64x16 * 128x16": A is 64x16 (M=64, K=16). B is stored as 128 columns of 16 values, so it is 16x128 (K=16, N=128). C is 64x128.

Note: the user said both namings at different times. The last message said A = weights, B = activations. This is also the naming in the original doc. The old code in `hw_sim.py` (earlier work) used the reverse. Use the naming above.

For a conv layer: `M = C_out`, `K = KH*KW*C_in`, `N = OH*OW*batch` (im2col).

2. The big idea in plain words

1. Most weights are zero and most activations are zero. Multiplying zeros wastes time.
2. **Packing** squeezes the zeros out. Take a slab of 4 K-positions. Two weight rows can share one packed row if they never have a nonzero in the same position. Same for activation columns. This is Mentha's trick.
3. Each packed element remembers where it came from with an **index** (original row/column number **plus 1**). Index **0 means "this element is zero, skip it"**. A weight with index 0 is a zero weight. An activation with index 0 is a mask that is all zeros (all 8 timesteps silent).
4. The array keeps the **weights still** and pushes the activations past them (weight-stationary).
5. Each PE has **8 small adders**, one per timestep. The 8 mask bits switch them on or off. So all 8 timesteps finish in one go. Time does not multiply the cost.
6. When the sum for an output is complete, a **one-cycle IF neuron** turns the 8 sums into 8 spike bits.
7. A **post-proc** unit unpacks the results back to the normal matrix and sends them to HBM.

What is different from Mentha, in three lines: weight-stationary, 8-timestep parallel adders, one-cycle IF. Everything else follows Mentha: index+value elements, graph-colouring packing, a threshold, skip-and-pass PEs, extra register-file slots in each PE, tiling done in memory.

3. Every stage, 0 to end

Stage 0: compile (once per model)

- Cut K into slabs of 4 (the array is 4 wide).
- Pack the weights of each slab (see 5). Store each packed element as `(weight, row index)`.
- Write one **descriptor** per pass: which slab, which chunk, how many packed rows R, K_eff, how many packed columns P, `final` flag (last slab), and the weight address.

Stage 1: HBM

Holds tiled matrices: packed weights, descriptors, raw activation masks, and a zero-filled output region. **Assumption from the user:** tiles are already laid out in HBM.

Stage 2: pre-proc (packs both operands)

- Weights come from the compiler. Activations (the previous layer's spikes) are packed here, per slab, with the same method.
- Assumption from the user:** packing takes no cycles. The packed tiles are already sitting in SRAM when the array needs them.
- An element is stored as $(\text{mask}, \text{column index})$ for activations and $(\text{weight}, \text{row index})$ for weights, with index = position + 1 inside the tile window.

Stage 3: load

A chunk of up to 4 packed weight rows is written into the PEs, one PE row per cycle ($\text{load_per_row} = 1$). Unused rows are switched off. Option $\text{shadow}=\text{True}$ hides this by loading the next chunk during the current pass.

Stage 4: stream

One packed activation column enters per cycle. Element k enters array column k , one cycle later than element $k-1$ (the **skew**). Then it moves down the column through every used row. Packed column (n') is at $\text{PE}(i,k)$ at cycle $(n' + i + k)$.

Stage 5: the PE

Each PE holds one weight and its row index. Each cycle:

- If the weight index is 0 or the activation index is 0 -> **skip and pass**. Nothing changes.
- Else look in the PE's **register file** (4 slots) for the pair $(\text{row index}, \text{column index})$. Found -> use that slot. Not found -> take a free slot.
- Add the weight into the 8 sums, only for the timesteps whose mask bit is 1.
- The set of slots moves right to the next PE. The activation moves down. Nothing waits.

Why 4 slots are enough: a set crosses only 4 PEs, and each PE adds at most one product, so a set never holds more than 4 pairs. (Mentha's own table agrees: slots = $\min(\text{threshold squared}, K)$.) The simulator checks this on every cycle. The biggest set seen was 4.

Stage 6: exit stage

- A set leaves the array at the last used column (column $(K_{\text{eff}} - 1)$).
- Entries go into a small FIFO. The exit stage serves up to $(\text{ports} = 8)$ entries per cycle.
- Not the last K-slab:** add the entry into the **partial buffer (PB)** at its (row, column). No IF yet. IF is nonlinear, so it must see the full sum over all of K .
- Last K-slab:** add the stored partial (if there is one), run the **one-cycle IF**, write $(\text{row}, \text{column}, \text{spike byte})$ into the **packed result buffer**.

- If the FIFO is nearly full, the array **freezes** for a cycle.

Stage 7: sweep

After the last slab: outputs touched in earlier slabs but not in the last one are still in the PB. They go through IF too (also `ports` per cycle).

Stage 8: post-proc

When the whole tile is done (`tile_done`), post-proc reads the packed result buffer and **unpacks** it: for each `(m, n, byte)` write `byte` at `(m-1, n-1)` of the dense spike tile (which starts as zeros). Then send the tile to HBM. The spike bytes are exactly the next layer's activation masks.

The IF neuron (matches the user's RTL, minus the leak)

`v += sum[t]; if v > theta: spike, v -= theta` for $t = 0..7$, all in one cycle. Threshold `theta >= 0` so an untouched zero never fires. (It is "IF with soft reset", not LIF, because there is no leak.)

4. Parameters

Name	Value	Meaning
Array	4 x 4	rows = packed weight rows (≤ 4 per pass), columns = the 4 K-positions of a slab
T	8	timesteps = bits per mask = adders per PE
Threshold (cap)	4	max originals merged into one packed row/column. 4 is the most that can ever merge (slab has 4 positions)
Slots per PE	4	register-file entries
Tile window	64 weight rows x 128 activation columns	indices fit in 7 and 8 bits. With 8-bit indices and 0 reserved, windows up to 255 are possible
Index	position + 1, 0 = zero	
Psum	16 bits per timestep in the array, 24 in the partial buffer	one slab sums at most $4 \times 128 = 512$. One psum entry = 8 timesteps x 16 bits = 128 bits (+ row idx, col idx, valid). Each PE holds 4 entries = 512 bits of sums. The sim asserts the 4-slot limit and the 16-bit range
Exit ports	8 entries/cycle	partial-buffer read-modify-write and IF lanes
FIFO depth	64 entries	array freezes if it could overflow
Weight load	1 PE row per cycle	
Post-proc	16 entries/cycle	only the last tile's unpack is counted (others overlap)
Clock	800 MHz	same as LoAS in <code>cycle_compare.py</code> . This is an assumption for Ours

Partial buffer size for a 64x128 tile: 8192 entries x (8 x 24 = 192 bits) = 192 KB. (An earlier version of this report said 128 KB, which assumed 16 bits per timestep. 24 bits is what the sim checks.) Cycle counts do not depend on these bit widths, only on entry counts and exit ports. Bit widths affect area and wiring: the exit moves 8 entries x 128 bits = 1024 bits per cycle.

5. How packing works (with a small example)

Weights A (4x4), rows 1..4:

row	k0	k1	k2	k3
1	3	0	0	0
2	0	0	-2	0
3	0	5	0	1
4	4	-1	0	0

Rows 3 and 4 clash (both nonzero at k1). The greedy grouping (highest clash-count first, then fill with non-clashing rows up to the cap) gives packed row `(idx [4,4,2,0])`, and packed row `(idx [1,3,0,3])`. This is exactly what `(snn_sim.py)` prints. An index of 0 is an empty spot.

Streaming one packed activation column through, the PEs make entries like `((3,7))`: row 3 times column 7. When the same pair meets again later in the row (weight 1 at k3 times column 7), it **matches the slot** and adds into it. IF on its 8 sums `([0,0,0,0,1,0,5,5])` with theta 2 gives spike byte `(0xC0)`. The simulator reproduces this and matches the dense answer.

6. Timing (derived, then checked in the simulator)

For one pass with `(P)` packed activation columns, `(R)` packed weight rows (≤ 4), slab width `(K)`:

- The last set reaches the exit at cycle index `(P + R + K - 2)`.
- So the pass takes `(P + R + K - 1)` cycles (plus `(R)` cycles to load the weights, plus any stall cycles).
- The simulator checked this with no stalls: R=1,P=1 -> 5; R=2,P=5 -> 10; R=4,P=20 -> 27; R=3,P=7 -> 13. All equal the formula.

Per tile: sum of all passes of all slabs, then the sweep, then +1 for the registered IF output.

Per layer: sum over tiles, plus the unpack tail of the last tile.

Passes per slab = `(ceil(R_slab / 4))`. Packing makes `(R_slab)` and `(P)` smaller, so there are fewer and shorter passes.

Control signals: `(act_ready)`, `(load_cnt)`, `(stream_cnt)` (counts to P), `(drain_cnt)` (R+K-1 after the last injection), `(final)` (from the descriptor), `(ent_valid)` (index not 0), `(result_val)` (launch IF), `(lif_done)` (result ready), FIFO full -> `(freeze)`, `(pass_done)`, `(sweep_start/done)`, `(tile_done)`. The next pass waits for the FIFO to be empty (drain barrier).

7. The example: 64x16 * 16x128 on the 4x4 array

K = 16 -> 4 slabs. One 64x128 output tile. Results from `run_example.py` (random data, output equals the dense reference every time):

weight density	activation non-silent	cycles	passes	packed rows / slab (of 64)	packed cols / slab (of 128)	stall cycles
1.0	1.0	9,409	64	64.0	128.0	0
0.5	0.5	4,268	38	35.2	69.8	300
0.3	0.3	2,307	22	21.0	43.8	248
0.1	0.1	564	12	10.0	18.8	8

Dense with no skipping is about 9,400 cycles, so packing gives about 2.2x at 50% density and about 17x at 10%. The 0.5 and 0.3 cases show stalls: at 8 exit ports per cycle the exit stage is already a bottleneck when packed sets are rich.

8. Throughput: why it is not 1 / latency

- **Latency** = time for one image. $\text{cycles}(1) / f$.
- **Throughput** = images per second when the machine works on a batch of B images: $B * f / \text{cycles}(B)$.
- They are equal only if $\text{cycles}(B) = B * \text{cycles}(1)$. That is true for LoAS in `cycle_compare.py`: its work is output neurons divided by 16 TPPEs, so doubling the batch doubles the time. It stays flat at 90.1 to 90.2 images/s (VGG16 INT8).
- It is **not** true for Ours. The batch adds columns to B ($N = OH * OW * B$). The weights sit in the PEs for a whole pass, so more columns reuse the same load and fill. Small layers (the last conv layers and the fully-connected layers, with N = 1 to 16) benefit most. VGG16 INT8: **37.5 images/s at batch 1, 53.6 at 2, 69.9 at 4, 80.9 at 8, 87.6 at 16, 90.9 at 32, 92.1 at 64.**
- The plot uses batch 16 for the throughput bars, and shows the sweep in the third panel.
- Both models run layers one after another (no overlap across layers or images).

9. Results against LoAS and what they mean

`loas_vs_ours.png`. Data: layer tables, sparsity table and the LoAS formula from the user's `cycle_compare.py`, used unchanged (`cycle_compare.py` is imported and called). T = 8 rows only, because Ours has 8 adders and an 8-bit mask. As in that file, LoAS uses the QCFS sparsity and Ours uses the PASC sparsity.

Latency (ms, batch 1) and throughput (images/s, batch 16):

Network	Prec	LoAS ms	Ours ms	Ours/LoAS	LoAS img/s	Ours img/s
AlexNet	INT8	4.55	4.91	1.08x	219.6	250.2
AlexNet	INT4	3.78	3.82	1.01x	264.6	312.3
AlexNet	Mixed	4.19	4.35	1.04x	238.6	275.9
VGG16	INT8	11.09	26.66	2.40x	90.2	87.6
VGG16	INT4	9.08	20.16	2.22x	110.2	112.2
VGG16	Mixed	10.21	23.53	2.30x	97.9	98.3
ResNet18	INT8	20.56	21.08	1.03x	48.6	53.2
ResNet18	INT4	16.50	16.92	1.03x	60.6	67.7
ResNet18	Mixed	18.73	18.94	1.01x	53.4	59.3

Why Ours does not win here (VGG16 INT8, batch 1, per layer, Ours / LoAS cycles)

- Big-N conv layers (N = 1024 and 256): 0.42x to 0.99x. Ours is as fast or faster.
- Middle layers (N = 64): 1.12x.
- Small-N layers (N = 16): 1.82x. (N = 4): 4.77x.
- Fully-connected layers (N = 1): 11.6x to 16x. These alone cost about 8 million of Ours' 21 million cycles.
- Reason: weight-stationary needs many activation columns to pay for each weight load and pipeline fill ($\lceil \frac{R + K}{4} - 2 \rceil$ extra cycles per pass). With N = 1, every 4-row weight chunk is loaded and used for one column.
- Second reason: at these densities (about 40% to 50% each) packing only shrinks rows by about 1.8x, so the sparse savings are small next to LoAS, which skips every non-match for free.
- Third: the exit stage (8 ports) stalls when sets are rich.

Sensitivity (VGG16 INT8, ms at batch 1 / images/s at batch 16)

Setting	ms	img/s
exit ports = 4	33.2	51.2
exit ports = 8 (default)	26.7	87.6
exit ports = 16	26.1	92.2
weight load hidden (shadow regs)	20.8	91.7
split: weights denser, activations sparser (wd = $p^{0.25}$, ns = $p^{0.75}$)	34.0	82.2
split: equal densities (wd = ns = $\sqrt{p}$, default)	26.7	87.6
split: weights sparser, activations denser (wd = $p^{0.75}$, ns = $p^{0.25}$)	23.0	84.7

Fairness check: same data for both

In the file, Ours sees PASC sparsity and LoAS sees QCFS. If Ours is given the **same QCFS numbers** as LoAS, it is 1% to 12% slower (for example VGG16 INT8: 26.96 ms instead of 26.66 ms; AlexNet INT8: 5.49 instead of 4.91). The conclusions do not change.

Note on the earlier plots in this chat

Earlier plots showed Ours 3x to 25x faster than LoAS/SATO/PTB. They used the LoAS paper's Table II statistics (about 98% of weights zero) and an older model of this design (merge cap 4, an unverified overlapped weight swap, online packing for activations). They are **stale and not comparable** with this report. Do not mix them.

10. Assumptions to keep in mind (these decide the numbers)

1. **Joint sparsity split.** `(cycle_compare.py)` gives only one number per config ("combined sparsity"). LoAS uses `(1 - combined)` as the chance that a weight/activation pair matches. I assume weights and activations are independent random with equal density: `(wd = ns = sqrt(1 - combined))`. The real split is unknown. The sensitivity rows show it matters (23 to 34 ms for VGG16 INT8). Real tensors may be correlated and structured, which could pack better or worse. **If real tensors exist, run the stepped simulator on them.**
2. Packing costs nothing and tiles sit in SRAM (user's assumption). Pre-proc time, HBM traffic and SRAM size are not charged.
3. 800 MHz for Ours (LoAS's number). The 8-step IF in one cycle may limit the clock.
4. Exit stage = 8 entries/cycle, FIFO of 64 (design choices, not from Mentha).
5. Weight load 1 PE row per cycle, no shadow registers by default. Mentha does not say how weights are loaded.

6. Partial sums across K-slabs are added by index in a partial buffer (192 KB per 64x128 tile at 8 x 24 bits). Mentha only shows a `C_in` input and does not say how partial results are stored. This is my filling of a gap.
 7. Drain barrier between passes (the back-to-back weight swap was never verified).
 8. Post-proc: only the last tile's unpack is counted.
 9. ResNet18 uses the 20-layer table in `cycle_compare.py`.
 10. The fast model samples random slabs. It matched the stepped simulator within 2.5% on average, 8.5% worst (very sparse small tiles).
-

11. How the design changed during the chat (so you can follow the history)

Start: the "initial doc" (Dual-Sparse Packed WS array): 1-bit tags, restore tables, online winQueue for activations, shadow weight registers, a merge unit plus accumulation buffer, LIF at the exit tap, threshold 2 per side.

Then, in order:

1. Both operands packed in pre-proc, offline-quality graph colouring. No winQueue.
2. Follow Mentha exactly except: weight-stationary, T=8 parallel accumulation, one-cycle IF.
3. Mentha-style `((index, value))` elements with **original** indices, no restore tables, no 1-bit tags. Index width fixed at design time from the window size (Mentha's equation 4: $\log_2$ of block size).
4. Threshold raised from 2 to 4 (4 is the most that can merge in a 4-wide slab). Slots stay 4 because a set only crosses 4 PEs.
5. Indices start at 1. Index 0 = zero element (Mentha section 3.2). With 0 reserved, an 8-bit index covers windows up to 255 (a 256 window needs 9 bits).
6. Tiling is done in HBM (Mentha partitions before the array).
7. Psums in the array are 16 bits per timestep (one slab sums at most 512). The 24-bit width is only for the partial buffer.
8. Order is array -> exit -> IF -> post-proc. Post-proc only unpacks. Because one pass only covers 4 K-positions, the exit stage keeps a partial buffer and runs IF on the last slab (then a sweep).
9. Naming settled: A = weights, B = activations.

Facts learned from the Mentha paper (pages read in this chat): packing is along the non-common dimensions only; blocks are `(M_blk x K_array)` with K_array equal to the array width; extra PE buffers are $\min(\text{threshold}^2, K)$ in its table; indices start from 1 and 0 marks a zero element; the paper does not describe loop order or the buffering of partial results across K tiles.

Problems found in the author's `Paper_Narrative.pdf`: operand names flipped; latency formula `((N-1)+M+(COLS-K))` leaves out the K columns the sum crosses and charges the

empty columns; applies IF at every exit with no K-slab handling; calls packing offline but uses a pre-proc unit during compute.

12. Open questions for the next step

- Does the real joint weight/activation density per layer exist? Replace assumption 1 with real tensors.
 - The fully-connected layers and the $N \leq 16$ conv layers dominate the loss. Ideas: batch more images (see throughput), a layer-wise choice of dataflow, or a 16-wide array for small N . Not tried yet.
 - Raise exit bandwidth (16 ports) and hide the weight load (shadow registers): both small gains (see sensitivity).
 - Can the 8-step IF fit in one 800 MHz cycle? Splitting it costs one cycle per tile.
 - How much pre-proc time and SRAM does packing really need?
 - Rewrite the earlier SATO and PTB baselines on this same data set if they are wanted again.
-

13. Files

File	What it is
<code>snn_sim.py</code>	The simulator. <code>python snn_sim.py</code> runs the self tests (hand example, pass-latency formula, random layers, 3-layer chain). Also holds the fast model
<code>run_example.py</code> , <code>example_output.txt</code>	The 64x16 * 16x128 example and its printed results
<code>validate_fast.py</code>	Fast model vs stepped simulator
<code>cycle_compare.py</code>	The author's LoAS script, unchanged
<code>compare_loas.py</code> , <code>results.json</code>	The network comparison and its raw numbers
<code>make_plot.py</code> , <code>loas_vs_ours.png</code>	The plot: latency, throughput, throughput vs batch
<code>REPORT.md</code>	This file

14. Addendum: "Ours should be 4x or more faster than LoAS" - checked, not supported by this data

The user expected a large win (LoAS is "slow"). I re-checked the model for unfairness and swept the data. Files: `sweep_sparsity.py`, `sweep.json`.

Speed-up = LoAS cycles / Ours cycles (above 1 = Ours faster), `cycle_compare.py` used unchanged for LoAS.

match density p (weight and activation both nonzero)	VGG16 b1	VGG16 b16	AlexNet b1	AlexNet b16	ResNet18 b1	ResNet18 b16
0.50	0.40	0.82	0.73	0.86	0.79	0.86
0.25 (about the table's INT8, T=8 value)	0.41	0.98	0.85	1.06	0.92	1.05
0.10	0.47	1.16	0.98	1.27	1.07	1.25
0.05	0.55	1.60	1.33	1.79	1.47	1.75
0.02	0.83	2.84	2.22	3.35	2.49	3.27
0.01	1.65	4.47	3.76	5.33	3.91	5.17

Second reading of the table (the docstring says N_matches = non-silent neurons, so weights dense INT8, p = activation density only): speed-up 0.36 / 0.94 (VGG16 b1 / b16), 0.82 / 1.15 (AlexNet), 0.90 / 1.10 (ResNet18).

Why it cannot be 4x at the table's sparsity (hardware-independent):

- LoAS in `cycle_compare.py` already runs all T timesteps in parallel (one cycle per match, for every timestep) and has 16 TPPEs. Our array has 16 PEs that also do one useful product per cycle, all 8 timesteps at once. So the "parallel" advantage is the same on both sides.
- LoAS costs $(\text{matches} + 21)$ per 128-element chunk. Even a perfect design (every PE busy every cycle, zero fill, zero stalls) can win at most $(p \cdot 128 + 21) / (p \cdot 128)$: 1.33x at p=0.5, 1.66x at p=0.25, 2.6x at p=0.1, 4.3x at p=0.05, 9.2x at p=0.02.
- A 4x or larger win needs the true match density near 5% or less. At the table's 72% to 86% combined sparsity (p = 0.14 to 0.28) it cannot happen.
- If the real weights are about 98% zero (as in the LoAS paper), p would be tiny and Ours would win big. Then the question is whether `cycle_compare.py`'s sparsity numbers describe that case. They do not look like it.

Open question for the author/prof: what exactly is "combined sparsity" in the table (activations only, or weights and activations together), and what is the real weight density?

15. LATEST RESULT: weights 90%+ sparse and activations 90%+ sparse (author's statement)

Setup: INT8 weights, 8-bit spike masks (T=8). Weight density = activation non-silent density = 0.10 (exactly 90% sparse; "90%+" means this is the least favourable case for Ours). LoAS uses `cycle_compare.py` unchanged with match density $p = 0.10 \times 0.10 = 0.01$ (weight-aware, so LoAS is not handicapped). Ours uses `snn_sim.py` (fast model, validated against the stepped simulator on sparse tiles: mean error 2.3%, worst 6.8%; the fast model now takes more random samples for narrow tiles because most slabs are empty). Plot: `loas_vs_ours_sparse90.png`. Raw numbers: `results_sparse.json`. Script: `compare_sparse.py`.

Network	LoAS ms (b1)	Ours ms (b1)	LoAS / Ours	LoAS img/s (b16)	Ours img/s (b16)	Ours / LoAS
AlexNet	2.05	0.62	3.3x	487	2,430	5.0x
VGG16	4.70	3.27	1.4x	213	885	4.2x
ResNet18	8.87	2.47	3.6x	113	546	4.8x

Gain grows with sparsity (batch-16 throughput, Ours / LoAS): 95% sparse -> 11.0x (AlexNet), 8.8x (VGG16), 10.6x (ResNet18). 98% sparse -> 26.2x, 21.2x, 25.4x. Batch-1 latency at 98%: 16.1x, 9.3x, 17.4x.

Throughput is derived as $\frac{B \times f}{\text{cycles}(B)}$, not $1/\text{latency}$. VGG16 at 90%: LoAS stays at 212.6 img/s for every batch. Ours: 306 (b1), 420 (b2), 569 (b4), 748 (b8), 886 (b16), 979 (b32), 1018 (b64).

Where Ours still loses (VGG16, 90%, batch 1, LoAS cycles / Ours cycles; below 1 = Ours slower): the first conv layers 4.9x to 15.7x faster; N=64 layers 3.4x; N=16 layers 1.36x; N=4 layers 0.48x; fully-connected layers (N=1) 0.26x to 0.31x. So the batch-1 latency gain on VGG16 is small (1.4x) because it has 3 big fully-connected layers and many small-N layers. Batching fixes most of it (see throughput).

Sensitivity (VGG16, 90%): exit ports 4 -> 3.68 ms / 622 img/s; ports 8 (default) -> 3.27 ms / 885 img/s; ports 16 -> 3.32 ms / 944 img/s; weight load hidden by shadow registers -> 2.43 ms / 1,030 img/s. So a shadow weight register file is the best single improvement (about 25% lower latency).

Assumptions that still matter: (1) "90% sparse activations" is read as 90% of the 8-bit masks being all-zero (non-silent density 0.10). If it means 90% of individual spike bits are zero, the non-silent density would be much higher and the gain smaller. (2) weights and activations

are independent random with those densities; real tensors may be structured. (3) the same assumptions as section 10 (packing free, 800 MHz, 8 exit ports, 1 PE row loaded per cycle). (4) the earlier table-based results (sections 9 and 14) used $p = 0.14$ to 0.28 , which only fits if the sparsity is much lower than 90%.